

Lecture 3: Failures (Why? Analysis of Systems)

CS 739: Distributed Systems

Prof. Andrea C. Arpaci-Dusseau
University of Wisconsin — Madison
Spring 2025

Today's Topic: Failures

Failure is the defining difference between distributed and local programming, so you have to design distributed systems with the expectation of failure. When you design distributed systems, you have to say, "Failure happens all the time." So when you design, you design for failure.

– Ken Arnold (Google Code University, Distributed System Design)

Jim Gray. [Why Do Computers Stop And What Can Be Done About It?](#)

HP Labs Technical Report TR-85.7 **SIGOPS Hall of Fame Award**

Aishwarya Ganesan, Ramnatthan Alagappan,

Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau,

[Redundancy Does Not Imply Fault Tolerance: Analysis of Distributed Storage Reactions to Single Errors and Corruptions,](#)

Proc. of 15th USENIX Conf. on File and Storage Technologies (FAST '17)

Why Today's Papers?

- Understand different reasons why systems fail (Heisenbugs)
- Advantages of fail-fast, modularity, redundancy
- Basic approaches of primary-backup
- Current distributed systems still cannot perfectly handle single faults!

What types of Failures?

Halting failures:

1. A temporal property of the system is violated (non-synchronized clocks, messages delayed)

Fail-stop:

2. Component stops with some notification to other components

Omission failures:

3. Messages or operations lost with no notification (e.g., no buffer space)

Network failures:

4. Component simply stops with no notification except by timeout (e.g., no heartbeat msg)

Network partition failure:

5. Data corruption or loss, failures caused by malicious programs

Timing failures:

6. A network fragments into two or more disjoint sub-networks

Byzantine failures:

7. A network link breaks

What types of Failures?

Halting failures:

Component simply stops with no notification except by timeout (e.g., no heartbeat msg)

Fail-stop:

Halting failure with some kind of notification to other components.

Omission failures:

Messages or operations lost with no notification (e.g., no buffer space)

Network failures:

A network link breaks

Network partition failure:

A network fragments into two or more disjoint sub-networks

Timing failures:

A temporal property of the system is violated (non-synchronized clocks, messages delayed)

Byzantine failures:

Data corruption or loss, failures caused by malicious programs

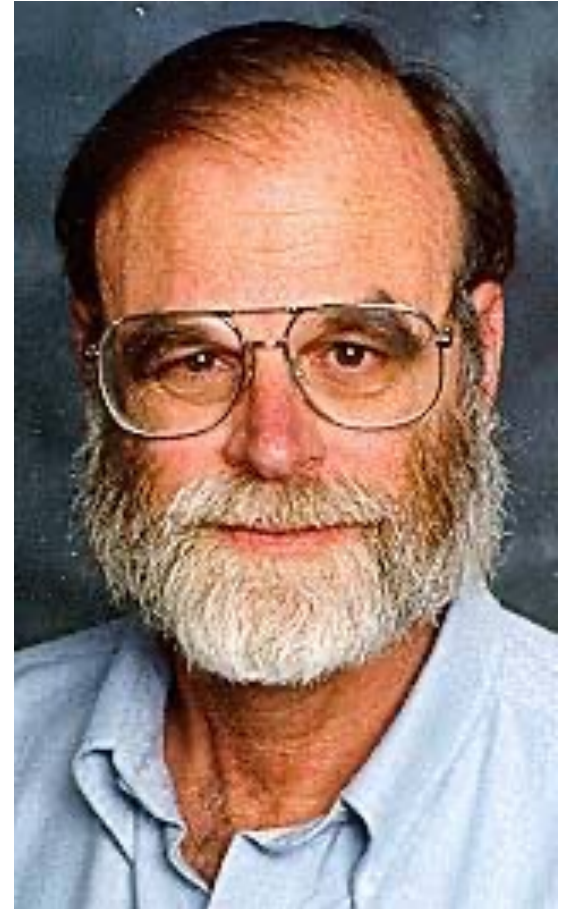
Who is Jim Gray?

Turing Award Winner, 1998

"for seminal contributions to database and transaction processing research and technical leadership in system implementation"

ACID, an acronym describing the requirements for reliable transaction processing and its software implementation

Disk-to-disk sorting benchmark!



Motivation?

- 1) Fault-tolerance is important in distributed systems
- 2) Understand causes of failures
- 3) Techniques for handling faults

Definitions

Reliability: paper has the definition

Availability: paper has the definition

$$\frac{\text{MTBF}}{\text{MTBF} + \text{MTTR}}$$

MTBF => Mean Time Between Failures
MTTR => Mean Time To Repair

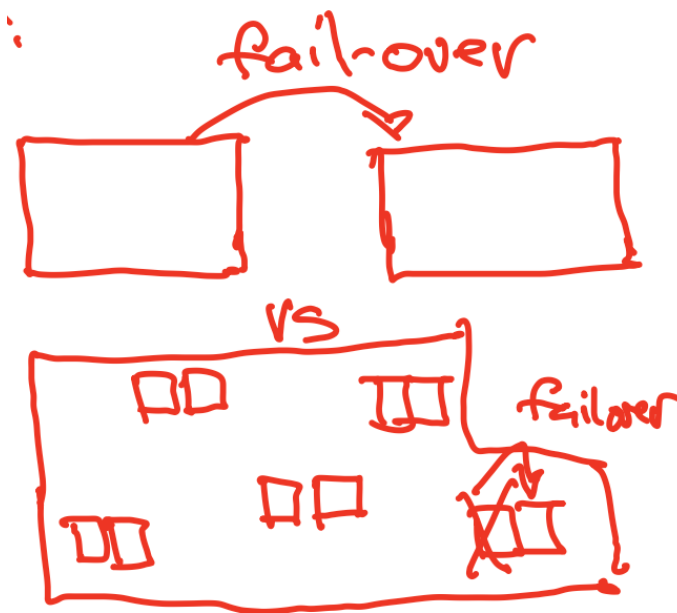
How to improve availability?

- 1) Increase MTBF => operate correctly for longer time
- 2) Reduce mean time to repair

How? auto failover, fail-fast approaches, do not want the failures to propogate to other components and spread it, hot spares

Keys: Redundancy, Modularity

Modularity => Reduce the state size so that a failure is limited to a module and the other modules can react accordingly.
Splitting the larger system to smaller pairs so that a fault in one of them => robustness is increased since the rest of the modules are unaffected.
Failovers are also made easy since we can have a standby spare module to failover to once the "primary" module fails.



Sources of Failures

From measurements, what percentage of failures caused by each?

Administration:

Software:

All this is in the paper

Hardware:

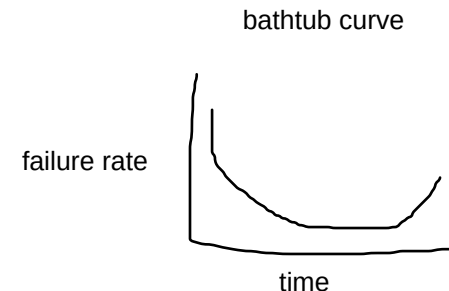
Environment:

Observations? Try to reduce manual interventions (administration is the cause of most problems => 42% acc to paper)

What are “infant mortality” failures?

Only for infant mortality failures

- new hw/sw causes a lot of faults in the beginning which should be ironed out in testing but might not be
- matured => bugs go down
- towards the end-of-life, support for new infant bugs go down



Contradiction?

New and changing systems have higher failure rates.

- Infant products and maintenance --> 2/3 of outages
- → Install proven hw and sw and **leave it alone**
- If it's not broken, don't fix it!

High percentage of outages are caused by “known” bugs

- Available (but not yet installed) fixes
- → Install fixes as soon as possible!

What to do? Compromise for the contradiction.

Hardware:

Install hardware fixes as soon as possible; needs maintenance

Software:

Install fix only if bug IS causing outages for you (workload and environemtn)

Obviously do urgent fixes like security patches

Gray's Principles of Fault-Tolerant Software

1) Software **modularity** through **processes + messages**

Processes have protection and fault boundaries

- Do not share state with others

Messages have well-defined interface

- Enables checking all inputs to process, control all state changes

Why is shared address space not good for fault-tolerance?

Shared address space fault => all processes sharing the space fails

Gray's Principles of Fault-Tolerant Software

This is more like a strategy in the protocol definition while definition

2) Fault containment through **fail-fast** software modules

Fail-fast: operate correctly, detect quickly so that propagation is not done much. Shutdown cleanly. This minimize data corruption and the radius of the failure

How to make component fail-fast?

- defensive programming
- check all input (asserts etc)

Assume perfect hardware? Can data be corrupted? No to both. We try to minimize the radius using fail fast.

How to detect another module has failed?

- heartbeats -- periodic RPCs and pings

Checkout redundancy paper, points to a software that still has the rest of the system functioning when the heartbeat component has failed.

- RPC timeout -- infer failed

Complications?

- The second point makes it difficult to differentiate between failed and slow.
 - => Lease
 - => Call itself dead if it's not able to talk to any other nodes

- Network Partitions. $A \leq X \Rightarrow B \Leftarrow \Rightarrow C$

There needs to be a policy for A to decide if itself dead or not. A common way to do this would be to see if A can talk to majority of the quorum of systems and decide if it's dead based on that.

Two Types of Bugs: Detection and Handling

1) Bohr bugs: Sold, reproducible, deterministic

We find them through standardized testing, code reviews.

They show up relatively easy => a workload that triggers this once, then it would happen the next time as well.

Handled?

2) Heisenbugs: Non-deterministic, race conditions, sensitive to ordering, timing between events.

Very hard to find, reproduce, eliminate. Hard to prove they don't exist.

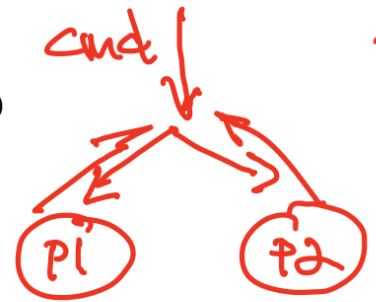
Probably won't happen again => retry the request/maybe retry on the other node

Handled?

Process-Pairs for HW + Transient SW Faults

a) lock-step?

It's assuming:
-fail-stop (won't work if one of them returns right + wrong, which one is right?)



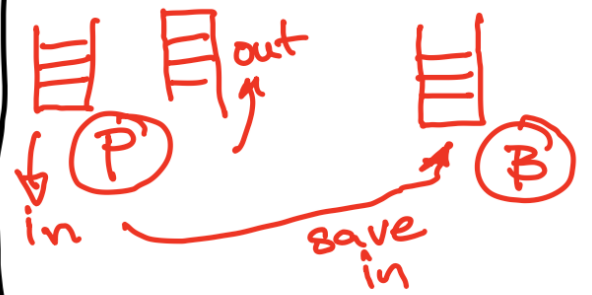
P1 fails?

Simple in practice

problem? Wasting resources
slow, synchronous

c) automatic checkpointing

Q? What if the state is stored in a 3rd place, a machine?

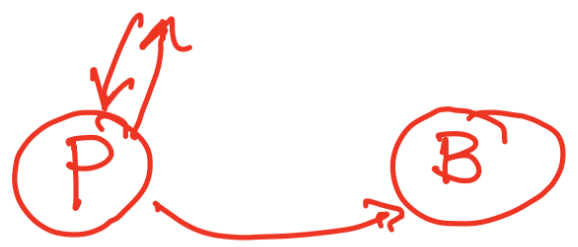


P1 fails?

Then there would be old commands run, slow and perf blip. Therefore, store state.

how to save msg space?
- periodic checkpoint/snapshot

b) state checkpointing



P fails?

state (periodically) + replies

problem? if the state is really big => slow to transmit checkpoint

d) delta checkpointing

With logging there's physical and logical:
- phy: bits changed
- logical: ops that need to be applied.



ops that need to be applied

Δ delta can be logical vs. physical (how did every bit that changed in mem)

advantages?

- 1) Smaller snapshot to send over the network
- 2) logical delta -- more robust to some Heisenbugs. Heisenbugs related to the memory are not sent over in this. The backup might have a better chance of not running into the Heisenbugs because of this.

Persistent Process Pairs



+ simple, efficient

Ⓟ
initial state

- what was happening??

Transactions:

Atomicity - all or none, commits or aborts

Consistency - sees correct state w/ concurrency

Integrity

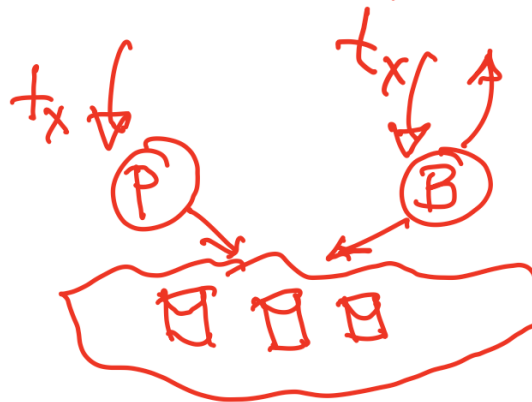
Durability - after commit, preserved

begin Tx

```
a++;  
b--;  
c = a + b;
```

end Tx

persistent process pairs + transactions



Fault-tolerant storage

Conclusions

Systems fail for many reasons (human error, sw, hw, environment)

Building fault-tolerant software systems benefits:

- Modularity and boundaries with processes and messages
- Redundancy with process pairs
- Fail-fast: Check data, defensive programming
- Tolerate Heisenbugs

Announcements

Reading Group 1: Meet with some members of reading group by Sunday

- Submit short individual write-up in Canvas
- Discuss past or future papers, could go over posted questions
- Will meet with same group for Week 2

Mini-Project 1 released later today: RPC for Client/Server KV Store

Focus only on sections listed

Monday: State and NFS – Focus on parts of NFS papers having to do with State → Design Implications for scalability, semantics, and recovery

1.State: [The Role of Distributed State](#) [Download The Role of Distributed State](#)(skip later sections on Sprite)

John Ousterhout, 1991

2.NFSv2 Background: [Design and Implementation of the Sun Network Filesystem](#) (Focus on material before Implementation)

Sandberg, R., Goldberg, D., Kleiman, S., Walsh, D., and Lyon, B., Proceedings of the Summer 1985 USENIX Conference, Portland OR, June 1985, pp. 119-130.

3.NFSv3: [NFS Version 3: Design and Implementation](#)[Links to an external site.](#) (Focus on material before Section 5: Performance)

Brian Pawlowski and Chet Juszczak and Peter Staubach and Carl Smith and Diane Lebel and Dave Hitz, In USENIX Summer 1994

4.NFSv4: [The NFS Version 4 Protocol](#) [Links to an external site.](#)(Focus on material before Section 10: Attributes)

Brian Pawlowski and Spencer Shepler and Carl Beame and Brent Callaghan and Michael Eisler and David Noveck and David Robinson and Robert Thurlow, Proceedings of the 2nd international system administration and networking conference (SANE2000)

This is to see how NFS evolved over the years.